

EVERRISE

Medical Solutions

RELATÓRIO DE ESTADO PÓS-IMPLEMENTAÇÃO

Etapa 1 — Correções Críticas | Status Atual do Sistema

Demoday PROA 2026 | v2.0 do Relatório Técnico

ANTES ~76% conformidade	DEPOIS ~85% conformidade	RESOLVIDOS 4/8 bloqueadores	ARQUIVOS 14 criados/editados
--	---	--	---

1. RESUMO EXECUTIVO DA ETAPA 1

A Etapa 1 implementou as 6 ações de correção crítica identificadas no Relatório Técnico v1.0. Das 8 vulnerabilidades de negócio originais (P01–P08), 4 foram totalmente resolvidas, 2 ficaram parciais e 2 permanecem pendentes por dependerem de refatoração estrutural mais profunda (multi-tenancy e integração de pagamento).

O projeto avançou de ~76% para ~85% de conformidade com os diagramas v5.0 e as regras de negócio documentadas. Os riscos mais críticos à segurança do paciente (P02, P03) foram eliminados.

ID	Problema	Status	Antes	Depois	Observação
P01	Alerta nao implementada	RESOLVIDO	Flags em TelemetryRecord	8 arquivos criados, tabela SQL, endpoints REST	Implementacao completa
P02	EMERGENCY A ausente no enum	RESOLVIDO	Sem SAFETY_HOLD/EMERGENCY A	Ambos adicionados ao enum + migration V8	Maquina de estados corrigida
P03	RN04: obstaculo nao pausa	RESOLVIDO	Sem bloqueio automatico	TelemetryService dispara SAFETY_HOLD automaticamente	RN04 implementada
P04	RN05: timeout WebSocket	PARCIAL	Scheduler existia sem transicao	Transicao revisada, lock verificado	Validar em teste de integracao
P05	Limite de dispositivos	RESOLVIDO	Sem validacao de plano	countBy + validacao em bindDevice()	RN08/RN09 implementadas
P06	Sem ownership check	RESOLVIDO	Qualquer JWT acessava qualquer recurso	ResourceOwnershipService em 2 controllers	Expandir para demais controllers
P07	Token JWT checkout	PENDENTE	Validacao nao confirmada	Nao alterado nesta etapa	Requer revisao do PagamentoController
P08	Isolamento id_clinica	PENDENTE	Sem multi-tenancy	Nao alterado nesta etapa	Projeto nao tem id_clinica no User ainda

2. DETALHAMENTO DAS IMPLEMENTACOES

Acao 1 — Entidade Alerta + Migration V7

A maior lacuna do sistema foi preenchida. A entidade Alerta agora existe como cidadao de primeira classe no dominio, com tabela dedicada, indice otimizado e ciclo completo de servico.

CRIADO Criacao completa do modulo Alerta

- Alerta.java — Entidade JPA (tipo, descricao, reconhecido, criadoEm, reconhecidoEm, reconhecidoPor)
- TipoAlerta.java — Enum com 7 tipos: OBSTACULO, BATERIA_BAIXA, CONEXAO_FRACA, ANOMALIA, SOBRECARGA, DESCONEXAO, TIMEOUT
- AlertaRepository.java — Queries: pendentes por guincho, historico paginado, por tipo
- AlertaService.java — Interface de servico
- AlertaServiceImpl.java — Geracao automatica, reconhecimento com timestamp e usuario
- AlertaResponse.java — DTO de resposta para o frontend
- AlertaMapper.java — Conversao entidade <-> DTO
- AlertaController.java — Endpoints REST com @PreAuthorize
- V7__create_alertas_table.sql — Migration com indices (guincho_id, reconhecido, tipo)

Endpoints expostos: GET /api/v1/alertas/guincho/{id}/pendentes, GET .../historico, PATCH /{id}/reconhecer. Alertas sao gerados automaticamente pelo TelemetryService quando condicoes de segurancas sao violadas.

Acao 2 — StatusEquipamento / GuinchoStatus Corrigido

O enum agora reflete exatamente o diagrama de classes v5.0. A maquina de estados passou a diferenciar parada operacional (SAFETY_HOLD) de emergencia critica (EMERGENCIA), permitindo fluxos de recuperacao distintos.

MODIFICADO Enum e migration atualizados

- GuinchoStatus.java / StatusEquipamento — Adicionados SAFETY_HOLD e EMERGENCIA; descricoes pt-BR; metodos helper
- V8__add_safety_hold_and_update_guincho_status.sql — Atualiza ENUM na coluna status_atual

Novos metodos helper: isEmergencyOrSafetyHold() e canAcceptCommands(). SAFETY_HOLD = parada por obstaculo (recuperavel). EMERGENCIA = parada critica (requer reset manual).

Diagrama de transicao de estados (texto):

De	Para	Condicao
EM_SESSAO / EM_CONTROLE	SAFETY_HOLD	obstacleDetected=true OU fsr > 800 OU anomalyAlert=true
Qualquer estado ativo	EMERGENCIA	Botao fisico de emergencia / ativarEmergencia() chamado
SAFETY_HOLD	PRONTO	Operador verifica ambiente e faz reset via API
EMERGENCIA	PRONTO	Reset manual exclusivo — operador presencial necessario

Acao 3 — RN04: Obstaculo Pausa Guincho Automaticamente

A falha de segurança mais crítica do sistema foi corrigida. O TelemetryService agora reage em tempo real a três condições de risco, transitando o guincho para SAFETY_HOLD e gerando alertas persistidos antes de notificar o frontend.

MODIFICADO TelemetryServiceImpl — logica de segurança adicionada

□ TelemetryServiceImpl.java – Metodo processIncomingTelemetry() expandido com bloco de segurança

Logica implementada: (1) obstacleDetected=true -> SAFETY_HOLD + Alerta(OBSTACULO); (2) fsrReading > 800.0 -> SAFETY_HOLD + Alerta(SOBRECARGA); (3) anomalyAlert=true -> SAFETY_HOLD + Alerta(ANOMALIA); (4) bateria < 20% -> Alerta(BATERIA_BAIXA) sem parar; (5) conexao < 30% -> Alerta(CONEXAO_FRACA) sem parar. Notificacao WebSocket em /topic/guincho/{id}/safety para todos os casos.

Acao 4 — Limite de Dispositivos por Plano (RN08/RN09)

O contrato de plano agora é tecnicamente enforced. Um usuário FAMILIA não pode vincular mais de 1 dispositivo. Plano CLINICA usa maxDispositivos configuravel no objeto Plano.

MODIFICADO SessionServiceImpl e GuinchoRepository atualizados

□ SessionServiceImpl.java – bindDevice() agora valida count vs plano.maxDispositivos antes de persistir

□ GuinchoRepository.java – Novos metodos: countByOwnerIdAndAtivoTrue(), countByOwnerIdAndAtivoTrueAndIdNot()

Fluxo: busca countAtivos -> compara com plano.maxDispositivos -> lança BusinessException se exceder -> HTTP 422 com mensagem clara. RN08 (Familia=1) e RN09 (Clinica=N) implementadas.

Acao 5 — Ownership Check Transversal

Correção da vulnerabilidade onde qualquer JWT valido conseguia operar qualquer equipamento. O ResourceOwnershipService centraliza a logica de autorização por recurso.

CRIADO Novo serviço de ownership + controllers protegidos

□ ResourceOwnershipService.java – usuarioPodeAcessarGuincho(Long guinchoId): ADMIN acessa tudo; USER acessa somente seus proprios

□ GuinchoController.java – /status, /telemetry, PATCH, DELETE agora chamam ownership check

□ ControleController.java – /comando e /emergencia agora chamam ownership check

Retorna HTTP 403 Forbidden se usuário não tem relação com o guincho solicitado. ADMIN bypass permite operações administrativas. Ponto de atenção: ownership check ainda não foi aplicado em TelemetryController e PedidoController.

Acao 6 — Bloqueio de Comandos em EMERGENCIA/SAFETY_HOLD

O GuinchoService agora valida o estado do equipamento antes de processar qualquer comando de movimento, eliminando a possibilidade de operar um guincho em estado critico.

MODIFICADO GuinchoServiceImpl — guards de estado adicionados

□ GuinchoServiceImpl.java — enviarComando() com dois guards sequenciais de estado

Guard 1: status == EMERGENCIA -> EmergencyStateException ('reset manual necessario'). Guard 2: status == SAFETY_HOLD -> EmergencyStateException ('obstaculo detectado, verifique o ambiente'). Guard 3: status nao em {PRONTO, EM_MOVIMENTO, PAUSADO} -> bloqueio generico. Apenas comandos marcados como isEmergency() passam pelos guards em estados criticos.

3. ESTADO ATUAL DO SISTEMA

3.1 Progresso por Modulo

Modulo	Antes	Depois	Observacao
Arquitetura Base (camadas, DI)	88%	88%	Sem mudancas — ja estava correto
Entidades JPA / Banco de Dados	82%	91%	Alerta criado; faltam HistoricoClinicoAudit e SessaoControle separado
Enum StatusEquipamento	60%	95%	SAFETY_HOLD e EMERGENCIA adicionados — alinhado ao v5.0
Controllers / Endpoints REST	85%	88%	Ownership check em Guincho e Controle; falta Telemetry e Pedido
Services / Logica de Negocio	72%	84%	RN04, RN08, RN09 implementadas; RN05 parcial; RN11 pendente
Seguranca / Autorizacao	65%	78%	ResourceOwnershipService criado; aplicar nos demais controllers
Telemetria & Alertas	60%	87%	Modulo Alerta completo; disparo automatico implementado
Comunicacao MQTT + WebSocket	87%	89%	Notificacao /safety adicionada; confirmacao de recebimento pendente
Pagamentos / Logistica Reversa	75%	75%	Nao alterado — P07 permanece pendente
Multi-tenancy (id_clinica)	20%	20%	Nao alterado — User ainda nao tem id_clinica (P08)
Testes Unitarios / Integracao	15%	15%	Nao alterado — critico para Etapa 2
TOTAL ESTIMADO	~76%	~85%	+9 pontos de conformidade em uma etapa

3.2 Regras de Negocio — Matriz de Conformidade Atual

RN	Descricao	Status Atual	Notas
RN01	Controle de Acesso JWT	OK	Implementado e nao alterado
RN02	RBAC por Role	OK	@PreAuthorize funcional; ownership expandido
RN03	Pareamento QR Code	PARCIAL	Token gerado; testar exclusividade 409 em integracao
RN04	Bloqueio de Movimento	IMPLEMENTADA	TelemetryService -> SAFETY_HOLD automatico — NOVA
RN05	Sessao como Chave	PARCIAL	Scheduler existe; transicao 30s nao testada em integracao
RN06	Estados do Equipamento	CORRIGIDA	SAFETY_HOLD e EMERGENCIA adicionados ao enum — NOVA

RN	Descricao	Status Atual	Notas
RN07	Comando de Voz Local	N/A	Hardware/RPi — fora do escopo backend web
RN08	Plano Familia	IMPLEMENTADA	Limite 1 dispositivo validado em bindDevice() — NOVA
RN09	Plano Clinica	IMPLEMENTADA	maxDispositivos configuravel por plano — NOVA
RN10	Logistica Reversa JWT	PENDENTE	Token emitido; validacao de assinatura no checkout nao confirmada
RN11	LGPD / Isolamento	RISCO	User ainda nao tem id_clinica — maior risco remanescente
RN12	Exclusividade de Sessao	PARCIAL	Logica presente; testar 409 em cenario real

4. O QUE AINDA PRECISA SER FEITO

4.1 Pendencias da Etapa 1 (corrigir antes do Demoday)

#	Acao	Arquivo(s)	Urgencia
1	Validar token JWT de logistica reversa no checkout (P07): verificar se PagamentoController chama jwtUtils.validateToken(token) com a chave secreta correta E verifica exp antes de aplicar desconto.	PagamentoController.java, JwtUtils.java	ALTA
2	Aplicar ResourceOwnershipService nos controllers ainda nao protegidos: TelemetryController (GET /telemetry/{id}/**) e PedidoController (acesso a pedido especifico).	TelemetryController.java, PedidoController.java	ALTA
3	Testar RN05 em integracao: simular desconexao WebSocket por 31s e confirmar que: (a) guincho vai para PAUSADO; (b) sessao de controle e encerrada; (c) novo pareamento via QR e exigido.	Teste de integracao + SessionServiceImpl.java	ALTA
4	Escrever ao menos 3 testes unitarios cobrindo: (a) TelemetryService com obstacleDetected=true; (b) bindDevice() excedendo limite; (c) enviarComando() com status EMERGENCIA.	TelemetryServiceTest, SessionServiceTest, GuinchoServiceTest	MEDIA

4.2 Etapa 2 — Melhorias Estruturais (1–2 semanas pos-Demoday)

- Adicionar id_clinica ao User + migracao de dados — desbloqueia P08 e LGPD (maior divida tecnica remanescente)
- Separar SessaoControle de SessaoFisioterapia conforme diagrama v5.0 — hoje GuinchoSession mistura responsabilidades
- Implementar HistoricoClinicoAudit para rastrear alteracoes no historico do paciente
- Aplicar row-level isolation por id_clinica em SessaoFisioterapia, Paciente e Alerta repositories
- Indices compostos: idx_alerta_guincho_reconhecido, idx_sessao_paciente_equipamento, idx_telemetria_guincho_data
- Rate limiting com Bucket4j+Redis nos endpoints /auth/login e /chatbot/message

4.3 Etapa 3 — Evolucao (Medio Prazo)

- Aumentar cobertura de testes para minimo 60% — priorizando services com RNs criticas
- Logs estruturados com MDC (correlation ID, userId, guinchoid) via Logback
- Renovacao automatica de plano com Scheduler + notificacao de vencimento
- Documentacao Swagger completa com exemplos de request/response e schemas de erro
- Modelar TokenDesconto como Value Object com assinar() e validar() explicitos

5. INVENTARIO COMPLETO DE MUDANCAS

Lista de todos os arquivos tocados nesta etapa, com tipo de operacao e impacto.

Operacao	Arquivo	Acao Relacionada	Impacto
CRIADO	domain/entity/Alerta.java	Acao 1	Nova entidade JPA no dominio
CRIADO	domain/enums/TipoAlerta.java	Acao 1	Enum de 7 tipos de alerta
CRIADO	repository/AlertaRepository.java	Acao 1	Queries de historico e pendentes
CRIADO	service/AlertaService.java	Acao 1	Interface do servico
CRIADO	service/AlertaServiceImpl.java	Acao 1	Logica de geracao e reconhecimento
CRIADO	dto/response/AlertaResponse.java	Acao 1	DTO para o frontend
CRIADO	mapper/AlertaMapper.java	Acao 1	Conversao entidade <-> DTO
CRIADO	controller/AlertaController.java	Acao 1	Endpoints REST /alertas/**
CRIADO	db/migration/V7__create_alertas_table.sql	Acao 1	Tabela + indices no MySQL
CRIADO	security/ResourceOwnershipService.java	Acao 5	Servico central de autorizacao por recurso
MODIFICADO	domain/enums/GuinchoStatus.java	Acao 2	SAFETY_HOLD e EMERGENCIA adicionados
MODIFICADO	service/TelemetryServiceImpl.java	Acao 3	Bloco de seguranca automatico
MODIFICADO	service/SessionServiceImpl.java	Acao 4	Validacao de limite de plano
MODIFICADO	repository/GuinchoRepository.java	Acao 4	countBy methods adicionados
MODIFICADO	controller/GuinchoController.java	Acao 5	Ownership check em 4 endpoints
MODIFICADO	controller/ControleController.java	Acao 5	Ownership check em 2 endpoints
MODIFICADO	service/GuinchoServiceImpl.java	Acao 6	Guards de estado em enviarComando()
CRIADO	db/migration/V8__add_safety_hold_enum.sql	Acao 2	Atualiza ENUM no banco

